



Tecnológico de Monterrey

Evidencia 1 - Reflexión

Daniel Gómez Guerrero A01572201

20 de septiembre del 2026

Estructura de Datos y Algoritmos Fundamentales (Gpo. 608)

David Alonso Cantú Delgado

Importancia de Ordenar

Es increíblemente importante saber o poder ordenar archivos de logs como los proporcionados en esta actividad por una variedad de razones. Entre ellas, destaca la seguridad de personas y entidades corporativas o federativas, ya que saber organizar logs permite encontrar con mayor facilidad los registros que marcan el acceso de individuos a todo tipo de páginas o lugares en el ciberespacio. Con esto, se puede construir una línea temporal estable que puede servir para identificar a ciberdelincuentes que entren a sitios restringidos en la web o que decidan robar datos importantes de personas alrededor del mundo. También es muy útil saber ordenar los logs para poder generar reportes de actividad en línea cuando sea necesario.

Comparación de los Algoritmos

Algoritmos	Fortalezas	Debilidades	Casos de Uso
Swap Sort - $O(n^2)$	Simple de implementar y relativamente fácil de entender.	Muy lento al ordenar datos, al realizar los intercambios uno por uno.	Útil para ordenar listas de datos pequeñas y simples.
Bubble Sort - $O(n^2)$	Relativamente simple de implementar y extremadamente rápido al ordenar archivos casi ordenados.	Bastante lento al ordenar archivos con datos mayormente desordenados.	Útil para ordenar listas de datos pequeñas o listas casi ordenadas.
Selection Sort - $O(n^2)$	Relativamente simple de implementar y disminuye significativamente la cantidad de intercambios necesarios.	Su tiempo a la hora de ordenar datos es solo moderadamente rápido y no varía demasiado entre listas casi ordenadas y desordenadas.	Útil para ordenar listas de tamaño moderado.
Insertion Sort - $O(n^2)$	Relativamente rápido al ordenar archivos casi ordenados.	Bastante lento al ordenar archivos grandes con datos desordenados.	Útil para ordenar archivos con pocos datos o con datos casi ordenados.
Quick Sort - $O(n \log n)$	Extremadamente rápido al ordenar archivos con datos desordenados y de	Significativamente menos rápido al ordenar archivos con datos casi ordenados	Especialmente útil para ordenar archivos desordenados con muchos datos.

	gran tamaño.	y algo difícil de implementar, además de requerir una cantidad significativa de memoria debido a su uso recursivo.	
Merge Sort - $O(n \log n)$	Extremadamente rápido al ordenar archivos tanto casi ordenados como desordenados.	Algo difícil de implementar y requiere una cantidad significativa de memoria debido a su uso recursivo.	Útil para ordenar archivos con grandes cantidades de datos y cuando se busca un rendimiento estable.
Shell Sort - $O(n \log n)$	Extremadamente rápido al ordenar archivos tanto casi ordenados como desordenados.	Su rendimiento no es tan estable como el de Merge Sort, teniendo variaciones un poco más significativas.	Útil para ordenar archivos con grandes cantidades de datos.

El Bubble Sort y el Quick Sort son los dos algoritmos de ordenamiento que más muestran un comportamiento fuera de la norma a la hora de ordenar, siendo que el Bubble Sort, a pesar de no ser muy eficiente, es extremadamente rápido al ordenar archivos casi ordenados, mientras que el Quick Sort, a pesar de ser un algoritmo del mismo nivel de complejidad que el Merge Sort y el Shell Sort, es significativamente más lento que ellos al ordenar archivos casi ordenados. Esto se puede deber a cómo funcionan sus métodos de ordenamiento, siendo que el Bubble Sort compara elementos consecutivos a la hora de realizar intercambios y, en un archivo ya casi ordenado, la cantidad de intercambios sería mínima. Por otro lado, el Quick Sort depende de la selección de pivotes y de la división de los datos en sublistas para comparar e intercambiar elementos, lo que puede resultar más lento si las particiones generadas no son eficientes al trabajar con elementos que ya se encuentran ordenados. Aunque tampoco se puede descartar la posibilidad de que los resultados obtenidos en las pruebas hayan sido influenciados por mi computadora o por la forma en que se desarrolló el código.

Interpretación de sus Mediciones

A pesar de que Swap Sort, Bubble Sort, Selection Sort e Insertion Sort tienen la misma complejidad teórica de $O(n^2)$, sus diferencias de eficiencia en tiempo fueron significativamente grandes. La cantidad de tiempo utilizada para ordenar un archivo completamente desordenado es casi tres veces mayor con Swap Sort que con Bubble Sort, y su diferencia al utilizarse en un archivo casi ordenado es incluso mayor, con una diferencia de más de 13,000 milisegundos. Esto demuestra que la complejidad teórica es solo una parte de lo que se debe considerar a la hora de determinar qué algoritmo de ordenamiento es más eficiente en cuanto a tiempo.

Ahora, eso no quiere decir que la complejidad teórica no sea útil para determinar la eficiencia de un algoritmo, pues es evidente en las pruebas que los algoritmos que tienen una complejidad de $O(n \log(n))$ suelen ser mucho más rápidos que los que tienen una complejidad de $O(n^2)$. Sin embargo, esto no siempre es cierto. Hay una gran excepción que demuestra que la velocidad al ordenar un archivo de logs no siempre es mayor con un algoritmo $O(n \log(n))$ que con uno $O(n^2)$, ya que Bubble Sort completa el ordenamiento de un archivo casi ordenado en 19 milisegundos (el tiempo más rápido entre los algoritmos utilizados), mientras que Quick Sort tardó alrededor de 4,000 milisegundos, un tiempo muy por encima de los demás algoritmos $O(n \log(n))$.

Esto demuestra que la forma en que funciona un algoritmo puede ser muy importante a la hora de identificar cuál es más útil para ordenar un archivo específico, pues hay ocasiones en las que un algoritmo de menor complejidad puede ser más eficiente que uno de mayor complejidad en ciertos casos.

Uso de IA

Durante la actividad, realmente no utilicé la Inteligencia Artificial de manera tan extensiva para realizar el código. Honestamente, recibí mayor ayuda al investigar en páginas como GeeksforGeeks y al preguntarle a mi hermano sobre el funcionamiento específico o recibir consejos sobre cómo utilizar algunas librerías y cómo hacer más eficiente mi código.

Sin embargo, sí utilicé la ayuda de Copilot para la creación de algunos de los comentarios en el código, aunque estos no siempre expresaban lo que realmente quería decir. Incluso llegó a colocar el mismo comentario, que no tenía nada que ver con el código, en múltiples secciones. Por esta razón, simplemente decidí modificar y editar varios de sus comentarios a mi manera.

Ahora bien, sí utilicé un par de prompts en ChatGPT para apoyarme en la creación de mi código. Específicamente, le pregunté: “¿Cuáles son los contenidos de las librerías `queue`, `sstream` y `fstream`, y para qué sirven?” Estas fueron librerías recomendadas por mi hermano que nunca había utilizado realmente, pero que me recomendó para la tarea. Esto me permitió tener un mayor entendimiento del funcionamiento de estas librerías, aunque aun así pedí confirmación de la información a mi hermano. Al final, no utilicé realmente `queue`, pero podría ser útil en un futuro proyecto o tarea.

Otro prompt que le hice varias veces a ChatGPT fue el siguiente: “Me está poniendo todos mis `cout` con error a pesar de usar `using namespace std`, ¿qué puedo hacer para solucionarlo?” Este fue un error que tomó algo de tiempo solucionar. Sin embargo, aun con la ayuda del chat, no estoy muy seguro de por qué ocurrió, pues al final simplemente se solucionó al remover el `namespace`, salir de Visual Studio y luego volver a ponerlo al regresar.

También le pedí a ChatGPT lo siguiente: “¿Podría revisar y arreglar solo los errores gramaticales de lo siguiente?” Esto fue para asegurarme de que lo escrito fuera del código estuviera correctamente redactado, ya que a veces tenía dudas sobre la gramática de secciones específicas.